

Lab 01 — Introduction to Cyber Defense

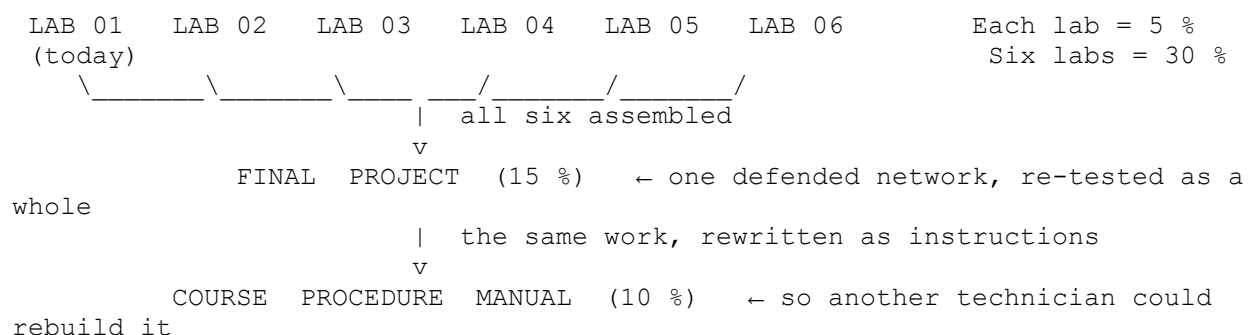
420-D03-AB · Week 01 · AEC (Attestation d'études collégiales) Cybersecurity,
LEA.D8 · Cohort CYB-01

Two content topics	(1) Defense Methods with AI tools · (2) Zoning Concepts with AI tools
Ponderation	2-3-3 (2 h theory · 3 h lab · 3 h personal work) — plan $\approx$ 3 hours on this lab
Weight	5 % of your final grade — one of six labs
Due	Sunday 13 September 2026, 23:59 (midnight) EDT
Submit to	The course Moodle → Lab 01 DropBox (one .zip archive)
Base files	The six Real-Life Example files from the Day 01 lecture (you already have them)

Read the last page first. The rules on the last page — *every tool here is swappable, be creative, and lawful use only* — change how you should approach everything in between. Two minutes now saves you a lot later.

1 · Why this lab matters — the ongoing project

This is not a throw-away exercise. **It is the first brick of a wall you will finish at the end of the course.** Here is the plan, and it does not change:



Every lab you do produces **four things**, every single time:

1. a **working configuration** (something that actually runs),
2. **screenshot evidence** (proof you did it),

3. a **test result** (proof it *works*, not just that it exists), and
4. a short **written finding** (what you learned, in your own words).

Keep all four, every week, in the folder you create in Step 4. At the end of the course you will *assemble* them into the Final Project and *rewrite* them into the Course Procedure Manual. **Students who keep up weekly finish calmly. Students who don't spend October trying to remember what a screenshot from September was for.**

The full end-to-end workflow is the one shown in the course **Content** overview on Moodle — this lab is session 01–02 of that map.

2 · Words you'll meet today

You are new to Information Technology (IT). That is expected. Here are the only terms you need for this lab, in plain language.

Term	What it means, simply
Host	Any single computer on a network — a laptop, a server, a printer.
IP address (Internet Protocol address)	A computer's number on the network, e.g. 192.168.1.10. Like a street address for a house.
Port	A numbered "door" on a host for one kind of traffic (port 443 = secure web). A host has thousands.
Service	A program on a host that listens on a port and answers requests (a web server, a file server).
Firewall	A guard that decides which traffic may pass between networks, by rules you write.
Zone / VLAN (Virtual Local Area Network)	A group of hosts that trust each other, kept separate from other groups by a control point.
LAN / WAN (Local / Wide Area Network)	Your inside network / the outside world (the Internet side).
Loopback 127.0.0.1	A special address meaning "this machine only." Traffic to it never touches a network card.
CLI (Command-Line Interface)	The text terminal where you type commands instead of clicking.
Script	A short text file of commands that runs them for you, the same way every time.
LLM (Large Language Model)	The kind of Artificial Intelligence (AI) behind chat assistants. It predicts text; it does not "know" things.

Term	What it means, simply
Ollama	A free program that runs an LLM entirely on your own computer — nothing leaves the machine.

Any other abbreviation in this document is spelled out in round brackets the first time it appears. Get in the habit of doing the same in your own writing — your marker will thank you.

3 · Your toolkit for today

You are **not** starting from a blank page. The Day 01 lecture shipped six small programs — the *Real-Life Examples* (RLE). This lab uses them as its base. You can run them straight from the interactive lecture page, or download them from its **Download Centre**.

File	What it does	Used in
rle01_cpm_scaffold.py	Builds your project folder + evidence register	Step 4
rle02_ai_triage_assistant.py	Local-AI alert summary with a privacy guard	Part A
rle05_defence_readiness_console.html	Scores how ready a network is	Part A
rle06_resilience_model.py	Calculates why layered defense wins	Part A
rle03_entry_point_inventory.sh	Lists every "door" a host is offering	Part B

One free tool to install: a local AI model. Install **Ollama** — <https://ollama.com> — then, in a terminal, pull a small model:

```
ollama pull llama3.2:3b
```

That model runs on a normal laptop with no special graphics card. If your machine is tight on memory, every AI step below also has an **offline** path that needs no model at all — you lose nothing essential.

The one rule for using AI in this course, on everything you submit: The model drafts. The analyst decides. The analyst's name goes on the record. An LLM writes fluent, confident text that is *sometimes wrong*. If you cannot point to the evidence for a claim yourself, the claim does not go in your report. Always say which tool and model you used, and what you changed after checking.

4 · Set up your project folder (≈ 10 min)

Run the scaffold script once, with your own name:

```
python3 rle01_cpm_scaffold.py --student "Firstname Lastname"
```

It creates a folder `CPM_Firstname_Lastname/` containing one sub-folder per lab, a seven-section write-up template in each, and an empty `EVIDENCE_REGISTER.csv`. **This is your Course Procedure Manual skeleton and your home for the whole course.** Open the cover file it made and add your student number.

Deliverable 4. A screenshot of the created folder tree. Save it into `LAB_01/evidence/`.

PART A — Defense Methods with AI tools (≈ 75 min)

A0 · The idea, in ninety seconds

Defense in depth means putting several *independent* controls on the same path, so that if one fails, the attacker is still not through. No single wall; many. A Security Operations Centre (SOC) — the team you are training to join — spends its day reading alerts and deciding which ones matter. That reading is slow and error-prone at 2 a.m., which is exactly where a **local** AI assistant earns its place: it drafts a plain-language summary, and a human verifies it. You will do both halves.

A1 · Local-AI alert triage with a privacy guard (≈ 45 min) — uses `rle02`

A real security alert is one of the most sensitive documents an organisation owns: it names hosts, accounts, internal addresses, and often the exact weakness that is currently unpatched. So it must **never** be pasted into a public chatbot. The script solves this two ways: it runs the model **locally** (on `127.0.0.1`, which never touches a network card), and it **redacts** dangerous fields *before* the model sees anything.

Do this:

1. Open the sample alert `sample_alert.json` and read it. Notice the `notes` field contains an e-mail address, a phone number, a pasted password, and a long code (a file hash). Three of those must never leave the building.
2. Run the tool **offline first**, so you see the safe baseline:
3.

```
python3 rle02_ai_triage_assistant.py --alert sample_alert.json --offline
```

Open the `decision_record.json` it produced. Find the field literally named `draft_not_verdict`. **Read that name — it is a control, not a decoration.**

4. Now run it **with your local model** and compare:
5. `python3 rle02_ai_triage_assistant.py --alert sample_alert.json`
6. Open the record again and check the four `analyst_*` fields. They are **empty on purpose** — the record is *structurally incomplete* until a human types a verdict and a name. Fill them in: read the raw alert, decide whether it is real, and write your one-line verdict and your name.
7. Prove the privacy guard works. In your own words, confirm that the password, e-mail, phone number, and hash from `notes` do **not** appear in the text that was sent to the model.

Deliverable A1. (a) Your completed `decision_record.json` with your analyst verdict and name filled in; (b) two–three sentences comparing the offline draft with the model draft — which read better, and did the model assert anything the alert did not support? *Finding that one over-reach is the actual skill.*

Make it yours. Ollama is one choice. Any local model runner (for example **Open WebUI** — <https://openwebui.com> — as a front-end) is fine. If you swap it, write down what you used and why (see the rules on the last page).

A2 · Measure the network, then reason about it (≈ 30 min) — uses `rle05` + `rle06`

An assistant is only useful if you can *measure* what it is helping with.

1. Open `rle05_defence_readiness_console.html` by double-clicking it (it works with the network unplugged — nothing is uploaded). Answer all fourteen questions **honestly for your own home network**. Read the final score, then **export the CSV**. The score is not the point; the rows that scored zero, in priority order, are.
2. Run the resilience model to see *why* several ordinary controls beat one excellent one:
3. `python3 rle06_resilience_model.py --scenario defended`

It prints the residual risk after each layer. Note the headline: seven ordinary layers leave far less residual risk than one strong control alone — because survival probabilities **multiply**, they do not add.

4. Ask your local AI to help you *explain* the result in plain English for a non-technical client — then **check its explanation against the numbers the script printed**. If the model invents a figure, that is the teaching moment.

Deliverable A2. (a) Your readiness CSV; (b) one short paragraph, in your own words, answering: *"Why is a stack of several 45 %-effective controls a better investment than one 90 %-effective control?"* — state the residual-risk figures, and name one **assumption** the model hides (hint: it assumes the layers fail independently, which is never fully true).

PART B — Zoning Concepts with AI tools (≈ 60 min)

B0 · The idea, in ninety seconds

A **zone** is a group of hosts that share a trust level, separated from other zones by a **control point that inspects the traffic between them**. Two networks with a router and an "allow everything" rule are **not** two zones — they are one zone with extra paperwork. Before you can zone a network, you must first know every **entry point**: every door a host is offering to the world. You cannot defend a door you never counted.

B1 · Count the doors (≈ 25 min) — uses `rle03`

1. Run the entry-point inventory on a machine you control (your own laptop, or the lab Linux virtual machine). It only **reads** — it changes nothing:
2. `bash rle03_entry_point_inventory.sh --out entry_point_register.csv`
3. Open the CSV. Each row is a listening service, classified by exposure: `ALL_INTERFACES` (reachable from every network the host is on — including guest Wi-Fi), `LOOPBACK_ONLY` (this machine only — safest), or `SPECIFIC_INTERFACE`.
4. **See your own action become a door.** In a second terminal, start a throw-away web server:
5. `python3 -m http.server 8080`

Re-run the inventory. Watch a new `ALL_INTERFACES` row for port 8080 appear. Stop the server (Ctrl-C) and you have just opened and closed an entry point on purpose.

Deliverable B1. Your `entry_point_register.csv`, plus one sentence per `ALL_INTERFACES` row answering: *"Who needs to reach this service, and from which zone?"* That single question is the whole of network defense, asked early.

B2 · Design the zones, let AI draft the rules, then verify (≈ 35 min)

Now you turn a flat network into a defensible one. Use this minimum zone plan for a small business as your starting point:

Zone	Contains	Rule of thumb
Perimeter	The firewall, the Virtual Private Network (VPN) gateway	Nothing else lives here
Servers	File server, backup target	Accepts connections from Users; starts almost none itself
Users	Staff laptops	Reaches Servers on named ports only
IoT (Internet of Things)	Printers, cameras, smart TVs	Reaches the Internet if it must; nothing internal
Guest	Visitor devices	Internet only — fully isolated from every other zone

Do this:

1. Draw this as a simple diagram (paper, or any free tool such as **draw.io** — <https://www.drawio.com>). Label the control point between each zone.
2. **Use AI as a drafting assistant, not an oracle.** Ask your local model to draft five plain-English firewall rules for the plan above — for example: *"Users may reach Servers on port 445; Guest may reach only the Internet; deny everything not explicitly allowed and log it."* Let it write a first draft.
3. **Now verify every line yourself**, because this is exactly where a model gets things subtly wrong. Check three things against the three rules of good zoning: (a) the **last** rule denies everything and logs it (default-deny); (b) each allow-rule names a **flow**, not a raw address, so it survives an address change; (c) no rule accidentally lets Guest reach an internal zone. Correct anything the model got wrong, and note what you corrected.

Deliverable B2. (a) Your zone diagram; (b) your final five firewall rules **after** your corrections; (c) two–three sentences naming at least one thing the AI draft got wrong or unsafe, and how you fixed it. If the draft was perfect, say how you *confirmed* that — "it looked right" is not verification.

Make it yours. The five-zone plan, the tool, and the example rules are all a **starting point**. A different, well-justified zone design can score *higher*. Research how real small networks are segmented and improve on this.

5 · Write one Course Procedure Manual entry (≈ 15 min)

Pick **one** thing you did today — creating the entry-point register is a good choice — and write it up in the seven-section template that is already sitting in your `LAB_01/` folder:

1. Objective · 2. Scope and assumptions · 3. Pre-conditions · 4. Procedure (numbered, one action per step) · 5. Verification (the test that proves it worked) · 6. Rollback (how to undo it) · 7. Evidence (file names, dates).

This is your first real Manual page. **The test it must pass:** a classmate who was not with you could follow it start to finish without asking you a single question.

Deliverable 5. One completed seven-section procedure.

6 · What to submit

Put everything into one **.zip archive** named `CYB01_LastName_FirstName_LAB01.zip` and upload it to the **Moodle → Lab 01 DropBox** by **Sunday 13 September 2026, 23:59 EDT**. Late work loses **10 % per calendar day** under department policy.

Your archive must contain:

- ☐ Folder-tree screenshot (Deliverable 4)
- ☐ Completed `decision_record.json` + your comparison note (A1)
- ☐ Readiness CSV + your residual-risk paragraph (A2)
- ☐ `entry_point_register.csv` + your one-sentence-per-door answers (B1)
- ☐ Zone diagram + corrected firewall rules + your "what the AI got wrong" note (B2)
- ☐ One seven-section Course Procedure Manual entry (Step 5)
- ☐ A one-line **AI declaration**: which tool and model you used, and what you changed after checking
- ☐ A short **change log** *if* you swapped any tool (what, and why)

Never submit real secrets. No real passwords, keys, or client data anywhere. If you must refer to a secret, name *what it is* and *where it is stored*, never the value itself. Before opening any external link, check it at **VirusTotal** — <https://www.virustotal.com/gui/home/url>.

7 · How this lab is marked (out of 5 %)

Criterion	Share	You earn it by
Both topics completed	30 %	All Part A and Part B deliverables present and runnable
Verification, not just configuration	25 %	Your evidence proves things <i>work</i> — you tested, you didn't only turn on
Judgement over the AI	20 %	You caught and corrected at least one thing the model got wrong
Evidence quality	15 %	Every file is named; every claim is tied to proof
Manual entry a stranger could follow	10 %	The seven-section write-up passes the classmate test

8 · The rules — read these, they matter

Everything here is open, and swapping is encouraged. Every tool, technique, address, and most of the content of these tasks is a **starting point, not a commandment**. If you find a better free tool, a cleaner method, or you want to **build your own script**, do it. The only condition: in your submission, **write what you changed from the original and why**. Research your tools thoroughly, compare alternatives, and be genuinely creative — that is the habit that makes an employable defender, and it earns marks rather than losing them.

This is a training simulation, on systems you own. Some tools and techniques in this course resemble the real ones used by real attackers and defenders. That resemblance is the point of professional training — and it carries responsibility. Everything in this lab is to be run **only** on a computer you personally own, or on the lab environment assigned to you, **never** on the college network, an employer's systems, or anyone else's device without **explicit, written** authorisation. This applies to every technique in this course, **including any not specifically named in this document**. If you are ever unsure whether an action is allowed, it is not — stop, and ask your instructor in writing first.

Lawful use only. Unauthorised access to, or interference with, a computer system is a criminal offence — in Canada under the **Criminal Code**, section 342.1 — <https://laws-lois.justice.gc.ca/eng/acts/c-46/> — and under equivalent laws elsewhere (the United States **Computer Fraud and Abuse Act**, 18 U.S.C. §1030; **European Union** Directive 2013/40/EU; **Spain's** Ley Orgánica 10/1995). Personal information you encounter is protected under Quebec's **Law 25** — <https://www.cai.gouv.qc.ca/> — and Canada's PIPEDA (Personal Information Protection and Electronic Documents Act). "Hacking back" is never permitted, regardless of provocation.

Free references worth your time. The **Canadian Centre for Cyber Security** — <https://www.cyber.gc.ca/en> — publishes plain-language guidance sized for small organisations. The **Center for Internet Security (CIS) Controls** — <https://www.cisecurity.org/controls> — list what to do first. **MITRE ATT&CK** — <https://attack.mitre.org> — catalogues how real attackers behave. The **NIST (National Institute of Standards and Technology) Cybersecurity Framework** — <https://www.nist.gov/cyberframework> — gives the vocabulary the whole industry uses.

This lab does not constitute professional or legal advice, and is provided for education only. It, and the choices in it, are your responsibility to apply lawfully.